



Numpy 2D en Python

Objectifs

Après avoir terminé ce lab, vous serez capable de :

- Manipuler `numpy` avec aisance
- Effectuer des opérations complexes avec `numpy`

Table des matières

- Créer un tableau Numpy 2D
- Accéder aux différents éléments d'un tableau Numpy
- Opérations de base

Créer un tableau Numpy 2D

```
In [ ]: # Importer les bibliothèques  
  
import numpy as np
```

Considérez la liste `a`, qui contient trois listes imbriquées (nested lists) **chacune de taille égale**.

```
In [ ]: # Créer une liste  
  
a = [[11, 12, 13], [21, 22, 23], [31, 32, 33]]  
a
```

Nous pouvons convertir (cast) la liste en un tableau Numpy (Numpy Array) comme suit :

```
In [ ]: # Convertir la liste en tableau Numpy  
# Chaque élément est du même type  
  
A = np.array(a)  
A
```

Nous pouvons utiliser l'attribut `ndim` pour obtenir le nombre d'axes ou de dimensions, appelé le rang (rank).

```
In [ ]: # Afficher les dimensions du tableau numpy

A.ndim
```

L'attribut `shape` renvoie un tuple correspondant à la taille ou au nombre d'éléments dans chaque dimension.

```
In [ ]: # Afficher la forme (shape) du tableau numpy

A.shape
```

Le nombre total d'éléments dans le tableau est donné par l'attribut `size`.

```
In [ ]: # Afficher la taille (size) du tableau numpy

A.size
```

Accéder aux différents éléments d'un tableau Numpy

Nous pouvons utiliser des crochets pour accéder aux différents éléments du tableau. La correspondance entre les crochets, la liste et la représentation rectangulaire est illustrée dans la figure suivante pour un tableau 3x3 :

A: $[A[0,0], A[0,1], A[0,2]], [A[1,0], A[1,1], A[1,2]], [A[2,0], A[2,1], A[2,2]]$

$$\begin{bmatrix} A[0,0] & A[0,1] & A[0,2] \\ A[1,0] & A[1,1] & A[1,2] \\ A[2,0] & A[2,1] & A[2,2] \end{bmatrix}$$

Nous pouvons accéder à la 2ème ligne, 3ème colonne comme indiqué dans la figure suivante :

	0	1	2
0	11	12	13
1	21	22	23
2	31	32	33

Il suffit d'utiliser les crochets et les indices correspondant à l'élément souhaité :

```
In [ ]: # Accéder à l'élément de la deuxième ligne et de la troisième colonne
A[1, 2]
```

Nous pouvons également utiliser la notation suivante pour obtenir les éléments :

```
In [ ]: # Accéder à l'élément de la deuxième ligne et de la troisième colonne
A[1][2]
```

Considérez les éléments illustrés dans la figure suivante

	0	1	2
0	11	12	13
1	21	22	23
2	31	32	33

Nous pouvons accéder à l'élément comme suit :

```
In [ ]: # Accéder à l'élément de la première ligne et de la première colonne
A[0][0]
```

Nous pouvons également utiliser le découpage (slicing) dans les tableaux numpy. Considérez la figure suivante. Nous aimerions obtenir les deux premières colonnes de la première ligne.

	0	1	2
0	11	12	13
1	21	22	23
2	31	32	33

Cela peut être fait avec la syntaxe suivante :

```
In [ ]: # Accéder à l'élément de la première ligne et des première et deuxième colonnes
A[0][0:2]
```

De même, nous pouvons obtenir les deux premières lignes de la 3ème colonne comme suit :

```
In [ ]: # Accéder à l'élément des première et deuxième lignes et de la troisième colonne
A[0:2, 2]
```

Correspondant à la figure suivante :

	0	1	2
0	11	12	13
1	21	22	23
2	31	32	33

Opérations de base

Nous pouvons également additionner des tableaux. Le processus est identique à l'addition de matrices. L'addition matricielle de **X** et **Y** est illustrée dans la figure suivante :

$$X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad Y = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

$$X + Y = \begin{bmatrix} 1+2 & 0+1 \\ 0+1 & 1+2 \end{bmatrix} = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$$

Les tableaux numpy sont donnés par X et Y

```
In [ ]: # Créer un tableau numpy X

X = np.array([[1, 0], [0, 1]])
X
```

```
In [ ]: # Créer un tableau numpy Y

Y = np.array([[2, 1], [1, 2]])
Y
```

Nous pouvons additionner les tableaux numpy comme suit.

```
In [ ]: # Additionner X et Y

Z = X + Y
Z
```

Multiplier un tableau numpy par un scalaire (scaler) est identique à multiplier une matrice par un scalaire. Si nous multiplions la matrice Y par le scalaire 2, nous multiplions simplement chaque élément de la matrice par 2, comme illustré dans la figure.

$$Y = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

$$2Y = \begin{bmatrix} 2 \times 2 & 2 \times 1 \\ 2 \times 1 & 2 \times 2 \end{bmatrix} = \begin{bmatrix} 4 & 2 \\ 2 & 4 \end{bmatrix}$$

Nous pouvons effectuer la même opération dans numpy comme suit :

```
In [ ]: # Créer un tableau numpy Y
```

```
Y = np.array([[2, 1], [1, 2]])
Y
```

```
In [ ]: # Multiplier Y par 2
```

```
Z = 2 * Y
Z
```

La multiplication de deux tableaux correspond à un produit élément par élément ou *produit de Hadamard* (Hadamard product). Considérez les matrices **X** et **Y**. Le produit de Hadamard correspond à la multiplication de chacun des éléments situés à la même position, c'est-à-dire multiplier ensemble les éléments contenus dans les boîtes de même couleur. Le résultat est une nouvelle matrice de la même taille que la matrice **Y** ou **X**, comme illustré dans la figure suivante.

$$X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad Y = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

$$X \circ Y = \begin{bmatrix} (1)2 & (0)1 \\ (0)1 & (1)2 \end{bmatrix} = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$

Nous pouvons effectuer le produit élément par élément des tableaux **X** et **Y** comme suit :

```
In [ ]: # Créer un tableau numpy Y
```

```
Y = np.array([[2, 1], [1, 2]])
Y
```

```
In [ ]: # Créer un tableau numpy X
```

```
X = np.array([[1, 0], [0, 1]])
X
```

```
In [ ]: # Multiplier X par Y (élément par élément)
```

```
Z = X * Y
Z
```

Nous pouvons également effectuer une multiplication matricielle (produit scalaire) avec les tableaux numpy **A** et **B** comme suit :

D'abord, nous définissons les matrices **A** et **B** :

```
In [ ]: # Créer une matrice A

A = np.array([[0, 1, 1], [1, 0, 1]])
A
```

```
In [ ]: # Créer une matrice B

B = np.array([[1, 1], [1, 1], [-1, 1]])
B
```

Nous utilisons la fonction numpy `dot` pour multiplier les tableaux ensemble.

```
In [ ]: # Calculer le produit scalaire (dot product)

Z = np.dot(A,B)
Z
```

```
In [ ]: # Calculer le sinus de Z

np.sin(Z)
```

Nous utilisons l'attribut numpy `T` pour calculer la matrice transposée.

```
In [ ]: # Créer une matrice C

C = np.array([[1,1],[2,2],[3,3]])
C
```

```
In [ ]: # Obtenir la transposée de C

C.T
```

Quiz sur les tableaux Numpy 2D

Considérez la liste suivante `a`, convertissez-la en un tableau Numpy.

```
In [ ]: # Écrivez votre code ci-dessous et appuyez sur Shift+Enter pour exécuter

a = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]]
```

► Cliquez ici pour la solution

Calculez la taille (size) du tableau numpy.

```
In [ ]: # Écrivez votre code ci-dessous et appuyez sur Shift+Enter pour exécuter
```

► Cliquez ici pour la solution

Accédez à l'élément de la première ligne et des première et deuxième colonnes.

```
In [ ]: # Écrivez votre code ci-dessous et appuyez sur Shift+Enter pour exécuter
```